황보민

원스토어 클라이언트 팀(9명) 기술 리드 · Android 개발자 · 팀 AI 하네스 구축
010-4354-1115 · 1stpie@naver.com · 경기 안양시

소개

원스토어 클라이언트 팀(9명)의 기술 리드이고, 통신사 단말에 기본 탑재되는 앱마켓 원스토어 앱(국내·글로벌)을 2020년부터 개발·운영해 왔습니다. 앱마켓의 핵심 경로인 다운로드·설치·자동 업데이트와 로그인·인증을 담당했습니다. 서비스 중인 두 앱에 5계층 Clean Architecture와 레이어별 모듈 구조를 도입하는 전환을 설계하고 직접 리팩토링하며 이끌었고, 화면 계층은 MVVM·MVI로, 비동기·DI는 Coroutine·Flow·Hilt로 바꿨습니다. 개발사가 쓰는 AAB Dynamic Delivery SDK를 설계·개발하고, 검증용 샘플 앱과 SDK 개발 가이드를 작성했습니다. 기술 리드로서 팀의 코드 리뷰 문화와 아키텍처 표준을 세우고 단위 테스트 체계를 만들었습니다. 2026년 1월부터는 팀의 AI 개발 하네스를 설계해 팀원 전원이 쓰게 했고, 테스트·코드 리뷰·아키텍처 검사를 자동화했습니다. SK텔레콤 선탑재 런처와 티스토어·원스토어 앱마켓 클라이언트를 개발해 온 경험으로 원스토어에 합류했습니다.

핵심 역량

- **앱 아키텍처·기반 기술 전환:** 서비스 중인 국내·글로벌 앱을 기능 개발과 병행해 단계별로 개편. 5계층 Clean Architecture와 레이어별 모듈 구조 도입, 레거시 구조를 MVVM·MVI로 변경, Coroutine·Flow·Hilt 도입, Service → WorkManager 전환, Compose 전환
- **SDK·개발 도구:** 개발사용 AAB Dynamic Delivery SDK 설계·개발(SDK와 앱 간 AIDL 규격, 검증용 샘플 앱·개발 가이드), 개발사 앱 APK·AAB 리패키징·재서명 도구 제작, Android View에 Box2D 물리엔진을 결합한 SDK 개발
- **모바일 앱 개발·운영:** 앱마켓(티스토어·원스토어)과 SKT 선탑재 런처 등 Android 앱을 개발·출시·운영, UI부터 Data까지 전 계층의 기능을 직접 개발, 제주항공 iOS(Swift) 앱 담당과 App Store 배포
- **기술 리드(2023~):** 팀의 PR·코드 리뷰 문화와 아키텍처 표준을 세우고 합의를 이끎, 기반 기술 도입 결정, 기술 전환 과제 주도, 팀 기술 학습 주도
- **팀 AX 개발·운영 환경 구축:** 팀 개발·운영을 위한 AI 하네스 구축(2026.01~). 룰과 에이전트 매뉴얼 작성, 업무의 스킬화를 통한 자동화, 훅·권한 기반 에이전트 제약, 업무 도메인별 에이전트와 개발 루프 설계, 하네스 최적화, 산출물 관리와 환각 억제를 위한 로컬 LLM Wiki 구축

경력 개요

회사	직함	기간	주요 일
원스토어	매니저 (클라이언트 팀 기술 리드) · 정규직	2020.03 ~ 현재	Android 앱(국내·글로벌) 개발·운영, 앱 아키텍처 전환 주도, 개발사용 AAB SDK 기술 리드(2023~), 팀 AI 하네스 구축(2026.01~)
AKIS	항공서비스팀 과장 · 정규직	2018.07 ~ 2020.03	제주항공 iOS(Swift) 앱 담당, 항공사 사내 앱 항공 예약 파트 3명 리드, 웹 개발 병행
미래ICT	프리랜서	2018.04 ~ 2018.06	현대모비스 차세대 내비게이터 선행 개발 참여 차량 마이컴(MCU) 연동 HIDL HAL 포팅(AOSP, C++)
이노에이스 (현 에스피테크놀러지)	모바일서비스 개발3팀 과장 · 정규직	2010.01 ~ 2017.10	앱마켓 앱(티스토어·원스토어) UI·비즈니스 로직 SKT 선탑재 런처·3D 런처, 3D UI 프레임워크(MUIF)

경력 상세

1. 원스토어 | 매니저 · 정규직 | 2020.03 ~ 현재

원스토어 앱은 통신사 단말에 기본 탑재되는 앱마켓 앱입니다. 국내·글로벌 앱의 대부분 영역 개발에 참여했고, 2023년부터 클라이언트 팀(9명)의 기술 리드를 맡고 있습니다.

기간	프로젝트	담당
2026.07 ~ 2026.10	웹게임 플랫폼 단말·광고 플랫폼 연동	웹게임 단말은 팀 과제(AI 하네스로 약 3주), 광고 플랫폼 연동과 개발사 테스트 환경 담당. 국내·글로벌 공용 광고 모듈 분리, 새 광고 서버 연동, 웹게임 권한 처리 기술: WebView, JavaScript 브릿지, 광고 SDK(Verse8·넵튠 ADX), 멀티 모듈, REST API 연동
2026.04 ~ 2026.09	2026 기술 부채 개선 과제 프로그램	진단 자동화와 구조 개선을 직접 진행. 앱 전체 의존성·아키텍처 진단을 하네스 스킬·매뉴얼로 자동화하고, 결과로 금지 참조와 레거시 설정 정리 기술: Clean Architecture, 멀티 모듈, Gradle, Claude Code 스킬
2026.03 ~ 2026.09	단위 테스트 체계 구축	테스트 작성 체계 설계와 작성 담당. 계층별 테스트 작성 방법을 가이드와 하네스 스킬로 만들어 에이전트가 대량 작성, Domain 계층 전체 단위 테스트, 계측 테스트 환경 복구 기술: Kotest(BehaviorSpec), MockK, Android 계측 테스트, Hilt, Claude Code 스킬
2026.01 ~ 현재	팀 AI 하네스 구축과 단계적 고도화	설계·구축·운영해 팀 전원이 사용. 룰·스킬·에이전트·훅·개발 루프·LLM Wiki 구축, 팀 개발을 AI로 진행하고 사람은 지시·감독·검수하는 체계로 전환, 구축 경험 회사 발표와 팀 교육 기술: Claude Code(플러그인·스킬·훅·에이전트), MCP, Maestro, Jenkins
2025.11 ~ 2026.01	미니게임 플랫폼 단말 개발	단말(앱) 쪽 개발 담당. 별도 프로세스 미니게임에 로그인·연령 정보 제공, 외부 연동과 홈 화면 바로가기, 광고 SDK 연동 기술: AIDL, Remote Service, ContentProvider, 다중 프로세스, 광고 SDK
2025.06 ~ 2025.09	다운로드·업데이트 구조 전환 (Service → WorkManager)	팀 과제의 핵심 전환을 직접 구현. 다운로드·자동 업데이트·설치 복구를 포그라운드 서비스에서 WorkManager로 옮기고 기존 서비스 제거 기술: WorkManager, Foreground Service, Hilt, Coroutine
2023.04 ~ 현재	글로벌 원스토어 앱 개발·운영	개발 참여, 출시 후 설치·자동 업데이트·비로그인 탐색 담당. 설치 모듈 분리, 파트너 설치 서비스 연동과 브랜드 앱 빌드, 비로그인 둘러보기 화면, 서버 호출 최적화 기술: Coroutine·Flow, Hilt, Room, Jetpack Compose, PackageInstaller
2022.09 ~ 현재	팀 기술 리딩·역량 개발	클라이언트 팀(9명) 기술 리드. 아키텍처 표준·코드 리뷰 문화 수립과 팀 합의, 기반 기술 도입 결정, 기술 전환 과제 주도, 팀 기술 세션 주제: 아키텍처, Coroutine·Flow, Hilt, 테스트, 코드 리뷰
2022.07 ~ 2025.09, 단계별	앱 아키텍처 전환과 리팩토링 (국내·글로벌)	아키텍처 설계와 단계별 전환 주도. 서비스 중인 국내·글로벌 앱에 5계층 Clean Architecture·모듈 구조 도입, 레거시 구조를 MVVM·MVI로 변경, 기능 개발과 병행 기술: Clean Architecture, MVVM·MVI, Hilt, Coroutine·Flow, 멀티 모듈
2021.11 ~ 2022.03	개발사 앱 APK·AAB 리패키징·재서명 도구	글로벌 스토어 개발사 앱을 다시 패키징·서명하는 도구를 개발. APK·AAB 리패키징·재서명, 서명 체계와 키 순환 정책 검토 기술: APK·AAB 구조, bundletool, apksig, APK 서명 체계
2021.09 ~ 2023	스티커스토어	스티커팩·이모지 구매·다운로드·편집 앱 개발·운영. RxJava·MVVM+ 기반, 원스토어에서 새 구조를 처음 적용한 앱 기술: RxJava, MVVM+, Retrofit·OkHttp, 수동 DI
2020.12 ~ 2024.10	팀 개발·QA 지원 도구	개발용 APK·APKS 다중 단말 설치 도구와 앱 검증 팀 지원 도구를 만듦 기술: bundletool, adb
2020.10 ~ 2021.10	AAB Dynamic Delivery SDK	기술 분석·설계와 SDK·앱 연동 개발. 개발사가 동적 기능 모듈·애셋 팩을 원스토어에서 내려받게 하는 SDK, 일부 기능 상용 적용 기술: Android App Bundle, Play Core, AIDL, bundletool, protobuf
2020.05 ~ 현재	다운로드·설치·자동 업데이트	다운로드·설치와 자동 업데이트를 담당해 개발·운영. 다운로드 요청·대기열·이어받기, 세션 설치와 사전 승인, 설치 결과 처리, 자동 업데이트, 설치 비호환 단말 관리 기술: PackageInstaller, Foreground Service, WorkManager, Coroutine, Room

기간	프로젝트	담당
2020.05 ~ 2026	로그인·회원 인증	로그인·회원 인증을 담당해 개발·운영. 성인·실명 인증, 로그인 잠금, 외부 앱·SDK 진입 로그인, 토큰 만료 시 자동 연장 기술: Coroutine, WorkManager, 토큰·세션 관리
2020.05 ~ 2025	원스토어 앱 화면 전반 개발·운영	화면 개발·운영 담당. 상품 전시(홈·카드·상세), 검색, My·설정, 구매 내역, 업데이트 목록·알림, Compose 전환 참여, 접근성 가이드라인 적용 기술: Retrofit·OkHttp, RecyclerView, Jetpack Compose, WebView, Android 접근성(TalkBack)
2020.04 ~ 2026, 단계별	앱 성능·안정성 개선	원인 분석과 개선 담당(기능 개발과 병행). 메모리 누수·ANR·전류 소모 문제 해결, 백그라운드 작업 방식 전환, 홈 화면 로딩 축소, 앱 시작 성능 계측 기술: WorkManager, Firebase Crashlytics, ANRWatchDog, Coroutine·Flow
2020.04 ~ 2020.09	원스토어 v7.0 앱 전면 개편	입사 직후 릴리스의 다운로드·인증·설정·안정화 영역 개발. 다운로드 흐름 재구현, 설정 화면, 서비스 분리, 메모리 누수 제거, QA 회귀 대응 기술: Android Service, WebView

1) 웹게임 플랫폼 단말·광고 플랫폼 연동 | 2026.07 ~ 2026.10

역할: 웹게임 플랫폼의 단말 쪽은 팀이 AI 하네스로 개발했고, 광고 플랫폼 연동과 개발사 테스트 환경은 본인이 맡았습니다.

한 일

- 웹게임(H5) 플랫폼의 단말 쪽을 팀이 약 3주에 개발(광고 연동 포함)
- 광고 플랫폼 연동: Verse8 광고 브리지와 넵튠 ADX를 연동하고, Verse8 전용이던 광고 실행 계층을 웹게임과 함께 쓰는 공용 모듈로 분리해 국내·글로벌 앱에 결합. 서명 기반의 새 광고 서버 규격으로 지면 조회와 토큰 등록을 전환
- 개발사 테스트 환경: 웹게임 개발사가 테스트 빌드를 실행하는 경로와 테스터 관리 화면을 만들고, 웹게임의 마이크·카메라 권한 요청을 국내·글로벌 앱이 대신 처리하게 함

기술: Kotlin, WebView, JavaScript 브릿지, 광고 SDK(Verse8·넵튠 ADX), 멀티 모듈, REST API 연동

2) 2026 기술 부채 개선 과제 프로그램 | 2026.04 ~ 2026.09

역할: 팀 과제에서 앱 전체 진단의 자동화와 구조 개선을 직접 진행했습니다.

한 일

- 진단 자동화: 모듈 의존성·아키텍처 규칙 위반·최소 지원 버전 아래 코드를 찾는 진단을 하네스 스킬·매뉴얼로 만들어 개선 대상 목록이 자동으로 나오게 함
- 구조 개선: 계층 경계를 어긴 참조를 없애고 데이터 소스 구현을 모듈 안으로 감췄으며, 재발을 막는 검사를 추가. 최소 지원 버전 아래의 코드·리소스·설정 정리

기술: Kotlin, Clean Architecture, 멀티 모듈, Gradle, Claude Code(스킬·에이전트)

3) 단위 테스트 체계 구축 | 2026.03 ~ 2026.09

역할: 테스트 작성 체계를 설계하고, AI 하네스로 테스트를 작성했습니다.

한 일

- 테스트 작성 체계: 계층별 테스트 작성 방법과 요구사항 형식을 정하고, UseCase·Repository 테스트 작성 가이드를 하네스 스킬로 만들어 여러 에이전트가 나눠 작성하도록 구성
- Domain 계층 UseCase 전체와 Repository 구현체의 단위 테스트를 작성하고, 테스트가 없던 곳을 찾아 보완하며 기능 개발 시 테스트를 함께 작성하는 규칙을 팀에 적용

- 계측 테스트 복구: 컴파일되지 않던 계측 테스트를 복구하고 Hilt·WorkManager 초기화 문제를 고쳐 실기기 테스트가 다시 돌게 함

기술: Kotlin, Kotest(BehaviorSpec), MockK, Android 계측 테스트, Hilt, WorkManager, Claude Code 테스트 작성 스킬

4) 팀 AI 하네스 구축과 단계적 고도화 | 2026.01 ~ 현재

역할: 설계·구축·운영했고, 클라이언트 팀(9명) 전원이 씁니다. 하네스는 코딩 에이전트에 팀 규칙·도구·검증 장치를 묶은 개발 환경입니다.

단계: 룰·스킬 체계(2026.01~) → 테스트 작성 체계 → Claude Code 이관과 팀 공통 플러그인 → 실제 과제 적용 → 개발 루프·리뷰·게이트 (2026.10 기준)

한 일

- 구성: 팀 룰과 에이전트 매뉴얼, 스킬 약 100개, 사내 MCP 서버로 Jira·Confluence·Bitbucket·Teams 연결, 역할별 에이전트(역할마다 도구 제한), 로컬 LLM Wiki 메모리. 팀 공통 플러그인으로 배포
- 통제·검증: 훅으로 파괴적 명령·보호 브랜치 push·외부 전송·무인 실행 중 질문 차단, 커밋·PR 규약 검사, push 전 테스트 게이트와 아키텍처 판정 검사
- 개발 루프: 한 줄로 시작하는 무인 실행이 요건 수집 → 실패 테스트 확인 → 구현 → 검증·리뷰 → 개선 순으로 돌고, 완료는 검증 결과로만 인정. 회차·시간 상한과 종료 조건, 종료 시 Teams 알림. 시작 승인과 최종 소스 검토는 사람
- 품질 자동화: Jenkins 단위 테스트 무인 정기 실행, Maestro·mobile-mcp 실기기 UI E2E, 데일리 코드 리뷰와 통합 리뷰, Crashlytics 원인 분석 → 승인 후 수정·PR, LeakCanary 누수 탐지
- 지식·자동화·공유: 산출물 관리와 환각 억제를 위한 로컬 LLM Wiki, Figma·Teams·Jira·Confluence 연동 업무 자동화, 주간보고·테스트 결과·리뷰 결과 공유 자동화, 팀 교육 가이드 작성과 회사 대상 발표

결과

- 팀원 전원이 이 하네스로 개발하고, 사람은 지시·감독·검수를 맡는 방식으로 팀 개발 방식이 바뀜
- 웹게임·광고 플랫폼 같은 새 영역을 빠르게 도입(웹게임 플랫폼 앱을 팀이 약 3주 만에 개발)하고, 도입 후 QA 검증 기간에 검출되는 이슈 감소

기술: Claude Code(플러그인·스킬·훅·에이전트), MCP, Cursor, Maestro·mobile-mcp, Jenkins, Firebase Crashlytics, SonarQube, LeakCanary, 로컬 LLM Wiki

5) 미니게임 플랫폼 단말 개발 | 2025.11 ~ 2026.01

역할: 원스토어 앱 안에서 설치 없이 실행하는 미니게임 서비스의 단말(앱) 쪽 개발을 맡았습니다.

한 일

- 별도 프로세스로 도는 미니게임이 로그인 정보와 연령 확인을 얻도록 AIDL·ContentProvider 인터페이스를 만들고, 외부 연동과 홈 화면 바로가기, 광고 SDK 연동과 의존성 충돌을 처리

기술: Kotlin, AIDL, Remote Service, ContentProvider, 다중 프로세스, 광고 SDK

6) 다운로드·업데이트 구조 전환 (Service → WorkManager) | 2025.06 ~ 2025.09

역할: 팀 과제로 진행했고, 핵심 전환을 직접 구현했습니다.

한 일

- 전환: 앱 다운로드, 통신사 업데이트, 외부 연동, 코어 업그레이드 등 다운로드 서비스가 맡던 경로를 Worker와 UseCase로 옮기고, 화면 이동·권한·네트워크 감시·설치 복구 로직을 서비스에서 분리한 뒤 서비스를 제거
- Android 제약 대응: 포그라운드 서비스 유형과 실행 시간 제한에 맞춰 Worker 종류별 시간 제한을 둠
- 배포 후 안정화
- 같은 시기 다운로드·설치 기능 리팩토링도 진행

결과: 국내 앱의 핵심 다운로드·설치·자동 업데이트 경로가 WorkManager로 옮겨지고 기존 백그라운드 서비스가 사라짐

기술: Kotlin, WorkManager, Foreground Service, Hilt, Coroutine, Clean Architecture

7) 글로벌 원스토어 앱 개발·운영 | 2023.04 ~ 현재

역할: 개발에 참여하고 출시 전 QA 이슈를 처리했으며, 출시 후에는 설치·자동 업데이트·비로그인 탐색 영역을 맡았습니다.

한 일

- 개발·출시: 국내 앱에서 설계한 새 구조를 글로벌 앱에 적용해 공통 UI 컴포넌트와 스토어 화면을 별도 모듈로 나누고, 로딩 오류 복구·보안 검사 같은 출시 전 안정성 작업과 QA 이슈 대응

- 설치 모듈: 설치 기능을 별도 모듈로 분리하고, 세션 설치·대체 설치·단말 내장 설치·파트너 SDK·Intent 설치를 바꿔 끼울 수 있는 구조로 만듦. 사전 승인을 설치 모듈에 위임하고 설치 실패·완료를 Firebase로 집계. 같은 구조를 국내 앱에도 적용
- 파트너 설치 서비스 연동과 게임 브랜드 앱: 파트너 설치 서비스의 인증·연결·설치 확인과 오류 처리, flavor별 패키지·서명·Firebase 설정으로 브랜드 앱 빌드. 토큰 갱신 문제를 파트너사와 함께 확인
- 비로그인 둘러보기: 로그인 없이 둘러보는 화면을 Compose로 만들고, 공통 JavaScript 인터페이스와 로그인 요청 규격을 구현. 웹뷰 미지원 단말 처리
- 서버 호출 최적화: 다운로드 인증 토큰 조회와 백그라운드 자동 업데이트 요청에서 불필요한 호출이 생기는 조건을 찾아 호출 조건과 스케줄을 바꿈

기술: Kotlin, Coroutine·Flow, Hilt, Room, Jetpack Compose, PackageInstaller, WorkManager, Firebase, Gradle productFlavors

8) 팀 기술 리딩·역량 개발 | 2022.09 ~ 현재

역할: 2023년부터 클라이언트 팀(9명)의 기술 리드를 맡고 있습니다. 아키텍처 개선 주도는 2022.07부터입니다.

한 일

- 기술 방향·표준: 새 아키텍처를 설계하고 계층별 모듈 구조와 개발 표준을 팀 합의로 확정. Coroutine·Flow·Hilt·WorkManager 등 기반 기술의 도입을 결정하고, Compose 전환을 팀에 전파
- 리뷰 문화·품질: 코드 리뷰 가이드를 정리해 팀 규칙에 반영하고 PR·리뷰 문화를 정착. 리뷰어로 팀원 PR을 검토하고 대규모 리팩토링 병합을 맡음. 계층별 테스트 작성 가이드를 만듦
- 표준을 지키게 하는 장치: 아키텍처 규칙 자동 판정 검사와 레이어 의존 위반 검사, 커밋·PR 규약 게이트, push 전 테스트 게이트를 만들고, 같은 규칙을 AI가 코드를 고칠 때도 적용해 사람과 AI가 같은 기준을 따르게 함
- 기술 과제 주도: Service → WorkManager 전환과 2026 기술 부채 개선 과제를 팀 과제로 이끎
- 팀 학습 주도: 새 구조를 샘플 아키텍처 프로젝트로 먼저 설명하고, 설계(MVVM·MVI·Clean Architecture·헥사고날 아키텍처·DDD·모듈화), 비동기·DI(Coroutine·Flow·Hilt), 테스트(테스트 코드·TDD), 앱 구조(APK·AAB, 서명 키 로테이션)를 주제로 팀 세션 진행. 본부 공유 여러 차례

9) 앱 아키텍처 전환과 리팩토링 (국내·글로벌) | 2022.07 ~ 2025.09, 단계별

역할: 2022.07부터 아키텍처 개선을 주도했고, 2023년부터는 기술 리드로서 이끌었습니다.

배경: 서비스 중인 앱에 MVC·일부 MVP 코드와 기술 부채가 쌓여, 서비스를 멈추지 않고 구조를 바꿔야 했습니다.

한 일

- 구조: UI·Presentation·Domain·Data·DataSource 5계층 Clean Architecture를 도입하고 앱 전체를 레이어별 모듈로 분리
- 설계 규칙: 계층 의존 방향을 UI → Presentation → Domain ← Data ← DataSource로 정해 Domain이 바깥 계층을 모르게 하고, 규칙 위반을 검사로 잡음
- 화면 계층: 레거시 구조(MVC·일부 MVP)를 MVVM + 단방향 데이터 흐름(Event-State 패턴)과 MVI로 바꾸고, Hilt 의존성 주입으로 계층 간 의존을 추상화
- 기반 기술: 비동기를 Coroutine·Flow로, DI를 Hilt로 전환. Compose 전환을 팀에 전파하고 상세·마이페이지 등 화면 전환에 참여
- 시범 적용: 스티커스토어 앱(2021.09~12)에 새 구조를 처음 적용
- 설계·시범(2022.07~2023.04): 샘플 아키텍처 프로젝트로 팀에 설명, 1차 모듈 분리와 domain의 Android 의존성 제거, 새 아키텍처 설계와 계층별 모듈 뼈대
- 글로벌 앱 리팩토링(2023.04~06): 이관받은 글로벌 앱을 레이어 분리와 모듈 구조로 정리
- 설치 모듈 분리·멀티 브랜드 빌드(2023.12~2024.03)
- 국내 데이터 계층 리팩토링(2024.01~07): Hilt를 도입해 데이터 매니저 싱글턴을 없애고, 콜백 기반 로더를 Flow UseCase로 대체, 데이터 계층을 Java에서 Kotlin으로 전환, 화면별 Manager를 Repository·UseCase·Flow로 이관
- UI 계층 리팩토링(2024.08~11)

- 다운로드·업데이트 구조 전환(2025.06~09)

결과: 서비스 중인 국내·글로벌 두 앱을 새 구조로 옮김

기술: Kotlin, Clean Architecture, MVVM·MVI, Hilt, Coroutine·Flow, Room, WorkManager, Jetpack Compose, 멀티 모듈

10) 개발사 앱 APK·AAB 리패키징·재서명 도구 | 2021.11 ~ 2022.03

역할: 글로벌 스토어에 올라오는 개발사 앱 바이너리를 다시 패키징하고 서명하는 도구를 개발했습니다.

한 일

- APK 서명과 AAB 리패키징 기능을 구현하고, bundletool·apksig로 매니페스트와 서명 체계(v1~v4)를 검증. 서명 키가 여러 개인 경우를 처리
- 만료된 키로 서명된 앱과 키 순환 후 업데이트 정책을 검토·시험. 이후 인앱결제 버전에 따라 서버 정보를 다르게 넣도록 보완

기술: APK·AAB 구조, bundletool, apksig, APK 서명 체계

11) 스티커스토어 | 2021.09 ~ 2023

역할: 스티커스토어 앱을 개발하고 운영했습니다.

한 일

- 스티커팩·이모지 구매·다운로드·편집 기능의 앱을 RxJava, MVVM + 단방향 데이터 흐름, Retrofit, 수동 DI 구조로 개발. 원스토어에서 새 구조를 처음 적용한 앱
- 운영: 이모지·스티커팩 로딩과 편집 화면의 결함 수정, 구매한 스티커팩의 다운로드 로직 수정
- 제휴사(삼성) 단말에서 생긴 문제의 원인이 라이브러리 난독화임을 찾아 해결 방안을 제휴사에 제시

기술: Kotlin, Java, RxJava, MVVM+, Retrofit·OkHttp, 수동 DI, ContentProvider, ProGuard/R8

12) 팀 개발·QA 지원 도구 | 2020.12 ~ 2024.10

한 일

- 개발용 APK·APKS(AAB 분할 APK)를 여러 단말에 한 번에 설치하는 도구(Mac·Windows)
- 앱 검증 팀의 업무를 돕는 도구를 만들어 Windows·macOS용으로 배포

기술: bundletool, adb

13) AAB Dynamic Delivery SDK | 2020.10 ~ 2021.10

역할: 기술 분석과 설계, SDK와 원스토어 앱 연동 개발을 맡았습니다.

한 일

- 개발사의 동적 기능 모듈과 애셋 팩을 원스토어에서도 필요할 때 내려받게 하는 SDK의 인터페이스와 기능을 정의
- SDK와 원스토어 앱 연동: 두 앱 사이의 AIDL 인터페이스와 전달 데이터 규격을 설계·구현하고, 화면 흐름, 로그인, 권한 확인, 강제 업데이트 조회, 오류 코드 처리
- 앱 종료 후 다운로드 오류, 네트워크 변경 시 이어받기, ANR을 해결하고 ProGuard 적용과 라이브러리 배포, AAB 미지원 단말 목록 관리
- 외부 개발자 지원: 검증용 샘플 앱과 SDK 개발 가이드 작성·갱신, 게임사 AAB 처리 오류 분석과 도구 버전 안내
- 단말 사양 수집: AAB 분할 전달의 전제가 되는 단말 사양 수집·등록 기능을 구현하고 자급제 단말의 수집 오류를 수정

결과: 원스토어 앱에 탑재되는 형태로 일부 기능을 상용 적용

기술: Android App Bundle, Play Core(AssetPackManager·SplitCompat), AIDL, bundletool, protobuf, ProGuard, Android 라이브러리 배포

14) 다운로드·설치·자동 업데이트 | 2020.05 ~ 현재

역할: 앱마켓의 핵심 기능인 다운로드·설치와 자동 업데이트를 2020.05부터 담당해 개발·운영하고 있습니다.

한 일

- 다운로드·설치: 다운로드 대기열·이어받기·망 전환, 세션 설치와 사전 승인, 설치 실패 진단과 결과 처리, Android 11·13의 설치·알림 변경 대응, 통신사앱 설치 지원

- 자동 업데이트: 원스토어·시드 앱의 자동 업데이트 Worker, 업데이트 요청 순서와 소유권 판정 정리, 불필요한 서버 요청이 생기는 조건을 찾아 호출 조건 정리
- 설치 비호환 단말 관리: 설치가 안 되는 단말을 서버와 동기화해 관리하고 사전 승인 생략·APK 대체 요청으로 설치 경로를 바꿈. 국내·글로벌 앱에 적용
- 설치 호환성 정리: 비선탑재(자급제) 단말의 설치 실패 원인을 찾도록 설치 시작 예외와 설치 제한 정책을 기록하게 하고, 저장 공간 로직을 분리

기술: PackageInstaller, Android Service·Foreground Service, BroadcastReceiver, ConnectivityManager, 알림 채널, WorkManager, Room, Coroutine

15) 로그인·회원 인증 | 2020.05 ~ 2026

역할: 로그인·인증·토큰 처리와 외부 앱·SDK에서 진입할 때의 로그인 흐름을 담당했습니다.

한 일

- 성인·실명 인증, 로그인 잠금, 본인인증 쿠폰 진입 흐름, AAB SDK·외부 앱에서 진입할 때의 로그인 시나리오
- 로그인 기능을 화면 클래스에서 분리해 상태와 동작을 나눈 구조로 재구성하고, 콜백 유실·중복 로그인 이벤트·캐시 순서 같은 비동기 결함을 수정
- 토큰 만료 시 자동 연장, 비로그인 화면의 로그인 유도와 복귀, 시드 앱 백그라운드 업데이트의 만료 토큰 관리

기술: Kotlin, Java, Coroutine, WorkManager, WebView, 토큰·세션 관리

16) 원스토어 앱 화면 전반 개발·운영 | 2020.05 ~ 2025

역할: 상품 전시·카드 영역을 담당했고, 화면부터 Presentation·Domain·Data·API 연동까지 기능 단위로 개발했습니다.

한 일

- 상품 전시(홈·패널·카드·상세): 카드 유형·배지·데이터셋 정책, 앱·게임·쇼핑 상세, 패널 갱신 개선
- 검색, My·설정·계정, 구매·결제 내역, 업데이트 목록·알림 화면 개발·운영
- 상세·마이페이지 등 화면의 Jetpack Compose 전환 참여, 접근성 가이드라인 적용

기술: Kotlin, Java, Coroutine·Flow, Lifecycle·ViewModel, Retrofit·OkHttp, RecyclerView, Jetpack Compose, WebView, Android 접근성(TalkBack)

17) 앱 성능·안정성 개선 | 2020.04 ~ 2026, 단계별

역할: 기능 개발과 병행해 시기별로 원인을 찾아 개선했습니다.

한 일

- 메모리 누수 제거(3일 방치 테스트), ANR 감시 도입, SDK 내부 Lock으로 생긴 ANR과 시스템 예외 누수 수정
- 삼성 단말의 전류 소모 VOC를 분석해 원인을 찾고 백그라운드 작업을 WorkManager로 전환, 초기화 순서로 생긴 크래시 수정
- 홈 화면의 중복 호출과 사전 로딩 축소, 오래된 메모리 대응 코드 제거, 앱 시작 성능을 TTID·TTFD 기준으로 계측해 구간별 소요 시간과 메인 스레드 차단을 분석하고 회귀 검증

기술: Android Service·Foreground Service, WorkManager, Firebase Crashlytics, ANRWatchDog, Coroutine·Flow

18) 원스토어 v7.0 앱 전면 개편 | 2020.04 ~ 2020.09

역할: 입사 직후 맡은 앱 전체 UI 개편 릴리스에서 다운로드·인증·설정·안정화 영역을 개발했습니다.

한 일

- 앱 다운로드 흐름 재구현(일반·연동 앱, 구매·성인 인증 연계, 상세 화면 다운로드 버튼)
- 웹뷰 기본 화면과 설정 화면, 허용 주소 검증 같은 보안 처리, 성인·실명 인증과 로그인 잠금

- 백그라운드 서비스 용도별 분리, 메모리 누수 제거, 화면 깜빡임·백화 개선, QA 회귀 대응과 v7.0.1 후속

기술: Java, Kotlin, Android Service, WebView

2. AKIS | 항공서비스팀 과장 · 정규직 | 2018.07 ~ 2020.03

제주항공 고객용 앱과 항공사 사내 앱 | 2018.07 ~ 2020.03

역할: 제주항공 고객용 iOS(Swift) 앱을 담당했고, 항공 예약 파트 구성원 3명을 리드하며 웹 개발을 병행했습니다.

한 일

- 고객용 iOS 앱 개발·운영, 예약 웹 화면과 연동되는 iOS 네이티브 브릿지 구현
- 고객용 앱 업데이트를 App Store에 배포하고 운영
- 항공 스케줄·정비 등 사내 앱 유지보수와 고도화, iOS 엔터프라이즈 인증서 기반 사내 배포

기술: Swift, WebView·JavaScript 네이티브 브릿지, App Store·엔터프라이즈 배포

3. 이노에이스 (현 에스피테크놀러지) | 모바일서비스 개발3팀 과장 · 정규직 | 2010.01 ~ 2017.10

앱마켓 앱 개발: 티스토어(원스토어 전신)·원스토어 | 2014.07 ~ 2017.10

역할: UI 레이어(컴포넌트·화면·라이프사이클·시스템 연동)와 사용자 비즈니스 로직을 담당했고, 고객사와 요건을 협의하고 시연했습니다.

한 일

- 티스토어 클라이언트 고도화: 전시·상세·검색·선물·멀티구매 화면과 서버·백그라운드 연동
- 원스토어 앱 개발과 고도화: 신규 상품 타입의 전시·상세·검색·선물·멀티구매 화면, 웹툰 상품 화면
- 원스토어 6.0 UI 전면 개편: 앱 대부분 화면의 UI 신규 개발, 서버 연동 모듈 수정·추가
- 선탑재 보조 앱 Utility·Setup Wizard 운영: 신규 화면, 고객 문제 대응

기술: Java, Android SDK, Glide, Volley, OkHttp

SKT 선탑재 런처와 3D 런처 | 2010.08 ~ 2014.03

역할: 런처의 Android 영역(화면, 엔진 렌더링 연동, 시스템 연동)을 개발하고 출시 후 운영했습니다. SmaDeco부터는 고객사와 요건을 협의하고 시연했습니다.

기간	영역	담당
2010.08 ~ 2011.12	폰꾸미기 (SKT 선탑재 런처, 13명 팀)	400만 이상 다운로드. MUIF 엔진 렌더링과 SurfaceView 연동, Lua 런처 컴포넌트, 테마 상점, 제휴사용 테마 제작 도구 기술: Java, Lua, SurfaceView, 3D 엔진 연동
2011.07 ~ 2012.07	Rabbit Launcher (3D 런처, 10명 팀)·SmaDeco (일본 현지화)	3D 엔진 렌더링과 SurfaceView 연동, 홈 기능, 테마 상점, 인앱 결제·포인트 서버 연동 기술: Java, SurfaceView, 3D 엔진 연동, 인앱 결제
2012.08 ~ 2014.03	T간편모드 (SKT 기본 탑재 런처, 4명 팀)	2G폰 키패드형 홈 UI, 시스템 권한 단말 설정 제어, 제조사 단말 포팅, 2013.04 상용 배포 기술: Java, Android 시스템 권한, 단말 포팅

기술: Java, Lua, SurfaceView, MUIF·3D 엔진 연동, Android 시스템 권한

그 밖의 프로젝트 | 2009.09 ~ 2014.07

기간	프로젝트	담당
2014.05 ~ 2014.07	SK Physics (Box2D SDK)	Android View와 Box2D 물리엔진을 연동하는 SDK를 2명이 함께 개발 기술: Java, Box2D
2014.05 ~ 2014.06	ARML (SKT AR 브라우저)	개발 참여: AR 화면 위 동영상 재생 기능 개발 기술: C++, OpenGL
2010.01 ~ 2010.07	MUIF (3D UI 프레임워크)	엔진 디버깅, 컴포넌트 SDK용 컴포넌트·WIPI-C 샘플 앱 제작 기술: C, WIPI-C, OpenGL 기반 UI 프레임워크

기간	프로젝트	담당
2009.09 ~ 2009.12	OMP 사업 (인턴)	앱마켓 플랫폼 사업에서 WIPI-C·Windows Mobile·Android 개발 경험 기술: WIPI-C, Windows Mobile, Android

기타 경력

- 미래ICT (프리랜서) | 2018.04 ~ 2018.06 | 현대모비스 차세대 내비게이터 선행 개발 참여: 차량 마이컴(MCU) 연동 HIDL HAL 포팅 (AOSP, C++)

학력·병역·자격

- **학력:** 단국대학교 컴퓨터과학과 학사(경영학 복수전공) (2003.02 ~ 2010.02)
- **병역:** 공군 병장 만기 제대 (2004.10 ~ 2007.01, 전산병: 웹 서버·DB·웹 애플리케이션 관리와 개발, 공군 전산 시스템 관리)
- **자격:** 정보처리기사 (2009)

기술

- **Android:** Jetpack(Lifecycle·ViewModel·Room·WorkManager·Paging·App Startup·Compose), Coroutine·Flow, RxJava, Hilt, Retrofit·OkHttp, RecyclerView, PackageInstaller, AAB, AIDL, WebView
- **품질·배포:** Kotest·MockK 단위 테스트, Android 계측 테스트, Maestro 실기기 UI 테스트, Firebase Crashlytics, Jenkins, 아키텍처 의존성 검증, Google Play·App Store 배포
- **언어·협업:** Kotlin, Java, Swift, C++, C, Lua · Git, Bitbucket, Jira, Confluence, Teams
- **AI 개발도구:** 팀 AI 하네스 구축(룰·스킬·에이전트·훅·개발 루프·로컬 LLM Wiki), Claude Code(플러그인·스킬·훅·에이전트), MCP, Cursor